

WEEK - 1 MODULE

1. Star Patterns

① Square

```
* * * * *  
* * * * *  
* * * * *  
* * * * *  
* * * * *
```

Approach:-

- Input the number of rows n .
- Use an outer loop to print rows.
- Use an inner loop to print n stars in each row.
- Move to the next line after every row.
- Repeat until all rows are printed.

Code:-

```
#include <iostream>  
using namespace std;  
int main()  
{  
    int n;  
    cin >> n;  
    for(int row=0; row<n; row++){  
        for(int col=0; col<n; col++){  
            cout << " * ";  
        }  
        cout << endl;  
    }  
    return 0;  
}
```

```
PS D:\Weekly modules> cd "d:\Weekly modules\" ; if ($?) { g++ squarePattern.cpp -o squarePattern } ; if ($?) { .\squarePattern }
```

```
5  
*****  
*****  
*****  
*****  
*****
```

```
PS D:\Weekly modules> |
```

② Hollow Square

Approach:-

- Use an outer loop to traverse rows.
- Use an inner loop to traverse columns.
- Print * when
 - Row is first ($row == 1$)
 - Row is last ($row == n$)
 - Column is first ($col == 1$)
 - Column is last ($col == n$)
- Otherwise print a space.
- Move to the nextline after each row.

Code:-

```
#include <iostream>
using namespace std;
int main()
{
    int n = 5;
    for (int row = 1; row <= n; row++) {
        for (int col = 1; col <= n; col++)
        {
            if (row == 1 || row == n || col == 1 || col == n)
            {
                cout << "*";
            }
            else {
                cout << " ";
            }
        }
        cout << endl;
    }
    return 0;
}
```

```
PS D:\Weekly modules> cd "d:\Weekly modules\" ; if ($?) { g++ hollowSquare.cpp -o hollowSquare } ; if ($?) { .\hollowSquare }
*****
* *
* *
* *
*****
```

③ Triangle.

1. Input the no. of rows n from the user.
2. Use an outer loop (row) to control the no. of rows.
3. Start the outer loop from 1 and run it up to n .
4. For each row, use an inner loop (col) to print stars.
5. Repeat the process until all rows are printed.

```
*  
* *  
* * *  
* * * *  
* * * * *
```

Code:-

```
#include <iostream>  
using namespace std;  
int main(){  
    int n;  
    cin >> n;  
    for (int row=1; row<=n; row++){  
        {  
            for (int col=1; col<=row; col++){  
                cout << " * ";  
            }  
            cout << endl;  
        }  
    }  
    return 0;  
}
```

```
PS D:\Weekly modules> cd "d:\Weekly modules\" ; if ($?) { g++ triangle.cpp -o triangle } ; if ($?) { .\triangle }
```

```
5  
*  
* *  
* * *  
* * * *  
* * * * *
```

④ Inverted triangle

- Input the no. of rows n .
- Use an outer loop to print each row.
- For every row, print $(row-1)$ leading spaces.
- Print $(n-row+1)$ stars.
- After printing a row, move to the next line.
- Repeat until all rows are printed.

```
*****
 *   *   *
  *  *  *
   * * *
    ***
     *
```

Code:-

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    for (int row = 1; row <= n; row++) {
        for (int space = 1; space < row; space++) {
            cout << " ";
        }
        for (int col = 1; col <= n - row + 1; col++) {
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}
```

```
PS D:\Weekly modules> cd "d:\Weekly modules\" ; if ($?) { g++ invertedTriangle.cpp -o invertedTriangle } ; if ($?) { .\invertedTriangle }
*****
 *   *   *
  *  *  *
   * * *
    ***
     *
PS D:\Weekly modules> 
```

⑤ Pyramid

- Input the no. of rows n .
- Use an outer loop to print each row.
- For every row, print $(n - \text{row})$ spaces.
- Print $(2 * \text{row} - 1)$ stars.
- Move to the next line after printing one row.
- Repeat until all rows are printed.

```
      *
     * *
    * * *
   * * * *
  * * * * *
 * * * * *
* * * * *
```

Code

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    for (int row = 1; row <= n; row++) {
        for (int space = 1; space <= n - row; space++)
            cout << " ";

        for (int col = 1; col <= (2 * row - 1); col++)
            cout << "*";

        cout << endl;
    }

    return 0;
}
```

```
PS D:\Weekly modules> cd "d:\Weekly modules\" ; if ($?) { g++ pyramid.cpp -o pyramid } ; if ($?) { .\pyramid }
```

5

```
*
**
***
****
*****
*****
*****
```

⑥ Diamond

- Input the number of rows n .
- Divide the pattern into two parts:
 - Upper Pyramid
 - Lower inverted pyramid
- Print the upper pyramid:
 - Spaces = $n - \text{row}$
 - Stars = $2 \times \text{row} - 1$
- Print the lower pyramid:
 - Spaces increases by 1 in each row.
 - Stars decrease by 2 in each row.
- Combine both parts to form a diamond shape.



Code

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    for (int row = 1; row <= n; row++) {
        for (int space = 1; space < n - row; space++)
            cout << " ";
        for (int col = 1; col <= 2 * row - 1; col++)
            cout << "*";
        cout << endl;
    }

    for (int row = n - 1; row >= 1; row--) {
        for (int space = 1; space <= n - row; space++)
            cout << " ";
    }
}
```

```

for (int col=1; col <= 2*row-1; col++)
{
    cout << "*";
}
cout << endl;
return 0;
}

```

```

PS D:\Weekly modules> cd "d:\Weekly modules\"; if ($?) { g++ diamond.cpp -o diamond }; if ($?) { .\diamond }
5
*
***
*****
*****
*****
***
*

```

⑦ Half Diamond

- Input the no. of rows n .
- Print the upper half
- Print the lower half
- Move to the next line after each row.
- Combining both halves forms the Half Diamond Pattern.

```

*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*

```

Code

```

#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    for (int row=1; row <= n; row++) {
        for (int col=1; col <= row; col++) {
            cout << "*";
        }
        cout << endl;
    }
    for (int row = n-1; row >= 1; row--) {
        for (int col=1; col <= row; col++) {
            cout << "*";
        }
        cout << endl;
    }
    return 0;
}

```

```

PS D:\Weekly modules> cd "d:\Weekly modules\"; if ($?) { g++ halfdiamond.cpp -o halfdiamond }; if ($?) { .\halfdiamond }
5
*
*
*
***
****
*****
****
***
*
*

```

⑧ Half Diamond inverted

```
*****
****
***
**
*
```

- Input the no. of rows n .
- Use an outer loop to print each row
- For every row, print $(row-1)$ leading spaces
- Print $(n-row+1)$ stars.
- After printing a row, move to the next line.
- Repeat until all rows are printed.

Code

```
#include <iostream>
using namespace std;
int main(){
    int n=5;
    for (int row=1; row <= n; row++){
        for (int space=1; space < row; space++){
            cout << " ";
        }
        for (int col=1; col <= n-row+1; col++){
            cout << "*";
        }
        cout << endl;
    }
    return 0;
}
```

```
PS D:\Weekly modules> cd "d:\Weekly modules\" ; if ($?) { g++ halfDiamondInverted.cpp -o halfDiamondInverted } ; if ($?) { .\halfDiamondInverted
}
5
*****
****
***
**
*
```

```
PS D:\Weekly modules>
```

2. Number Patterns

① Square

- Input the size n .
- Use an outer loop for rows.
- Use an inner loop for columns.
- Print number 1 in every position.
- Move to the next line after each row.

```
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
```

Code

```
#include <iostream>
using namespace std;
```

```
int main(){
```

```
    int n=5;
```

```
    for(int row=1; row<=n; row++){
```

```
    { for(int col=1; col<=n; col++){
```

```
        { cout << "1";
```

```
        }
```

```
        cout << endl;
```

```
    }
```

```
    return 0;
```

```
}
```

```
PS D:\Weekly modules\Number Patterns> cd "d:\Weekly modules\Number Patterns\" ; if ($?) { g++ square.cpp -o square } ; if ($?) { .\square }
```

```
11111
```

```
11111
```

```
11111
```

```
11111
```

```
11111
```

② Incrementing

- Input the size n .
- Use an outer loop for rows.
- Use an inner loop from 1 to n .
- Print the column no.
- Move to the next line after each row.

```
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
```

Code

```
#include <iostream>
using namespace std;
int main()
{
    int n=5;
    for (int row=1; row<=n; row++)
    {
        for (int col=1; col<=n; col++)
        {
            cout << col << " ";
        }
        cout << endl;
    }
    return 0;
}
```

```
PS D:\Weekly modules\Number Patterns> cd "d:\Weekly modules\Number Patterns\"; if ($?) { g++ incrementing.cpp -o incrementing }; if ($?) { .\incrementing }
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
```

③ Right triangle

- Initialize the no. of rows n .
- Use an outer loop (row) to control the rows from 1 to n .
- For each row, use an inner loop starting from 1.
- Print the value of col in every iteration.
- Repeat the process until all rows are printed.

```
#include <iostream>
using namespace std;
```

```
int main() {
    int n=5;
    for (int row=1; row<=n; row++) {
        for (int col=1; col<=row; col++) {
            cout << col << " ";
        }
        cout << endl;
    }
    return 0;
}
```

```
PS D:\Weekly modules\Number Patterns> cd "d:\Weekly modules\Number Patterns\"; if ($?) { g++ rightTriangle.cpp -o rightTriangle }; if ($?) { .\rightTriangle }
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

④ Inverted Diamond

- Initialize the number of rows $n=5$.
- Use an outer loop (row) from 1 to n .
- For each row, print spaces equal to the current row number.
- After printing spaces, print the number (row+2).
- Print the number (row+2) exactly $(n-\text{row}+1)$ times.
- Move to the next line after completing each row.
- Repeat the process until all rows are printed.

```
3 3 3 3 3
4 4 4 4
5 5 5
6 6
7
```

Code

```
#include <iostream>
using namespace std;

int main(){
    int n=5;
    for(int row=1; row <= n; row++){
        for(int space=1; space <= row; space++){
            cout << " ";
        }
        for(int col=1; col <= n-row+1; col++){
            cout << row+2 << " ";
        }
        cout << endl;
    }
    return 0;
}
```

```
PS D:\Weekly modules\Number Patterns> cd "d:\Weekly modules\Number Patterns\" ; if ($?) { g++ invertedDiamond.cpp -o invertedDiamond } ; if ($?) { .\invertedDiamond }
```

```
3 3 3 3 3
4 4 4 4
5 5 5
6 6
7
```

```
PS D:\Weekly modules\Number Patterns>
```

⑤ Sandwich Pattern

- Set the no. of rows n .
- Divide the pattern into two parts.
- In the upper part:
 - Print leading spaces equal to $(row-1)$.
 - Print the no. $(row+2)$.
 - The count of numbers follows the sequence 5, 3, 1.
- In the lower part:
 - Print leading spaces decreasing from $(n-2)$ to 0.
 - Print the no. in reverse order 4, 3.
 - The count of no. follows the sequence 3, 5.
- Move to the next line after each row.

```

3 3 3 3 3
  4 4 4
    5
  4 4 4
3 3 3 3 3

```

Code

```

#include <iostream>
using namespace std;
int main() {
    int n=3;

```

```

    for (int space=1; space < row; space++) {
        cout << " ";
    }
    for (int col=1; col <= 2*(n-row)+1; col++) {
        cout << row+2 << " ";
    }
    cout << endl;
}

```

```

for (int row=n-1; row >= 1; row--) {
    for (int space=1; space < row; space++) {
        cout << " ";
    }
    for (int col=1; col <= 2*(n-row)+1; col++) {
        cout << row+2 << " ";
    }
    cout << endl;
}
return 0;
}

```

```

PS D:\Weekly modules\Number Patterns> cd "d:\Weekly modules\Number Patterns" ; if ($?) { g++ sandwichPattern.cpp -o sandwichPattern } ; if ($?) { .\sandwichPattern }
33333
 444
 5
 444
33333

```

⑥ Incrementing Diamond pattern

- Set the number of rows n .
- Divide the pattern into two parts
- In the upper part, print the no. $(row+2)$ repeatedly row times.
- In the lower part, print the no. $(row+2)$ repeatedly row times in reverse order.
- Print a new line after each row.
- Combine both parts from a diamond-like pattern.

```

3
4 4
5 5 5
6 6 6 6
7 7 7 7 7
6 6 6 6
5 5 5
4 4
3

```

Code

```

#include <iostream>
using namespace std;
int main(){
    int n=5;
    for (int row=1; row<=n; row++){
        for (int col=1; col<=row; col++){
            cout << row+2 << " ";
        }
        cout << endl;
    }
    for (int row=n-1; row>=1; row=row-1){
        for (int col=1; col<=row; col++){
            cout << row+2 << " ";
        }
        cout << endl;
    }
    return 0;
}

```

```

PS D:\Weekly modules\Number Patterns> cd "d:\Weekly modules\Number Patterns\"; if ($?) { g++ incrementingDiamondPattern.cpp -o incrementingDiamondPattern }; if ($?) { .\incrementingDiamondPattern }
3
4 4
5 5 5
6 6 6 6
7 7 7 7 7
6 6 6 6
5 5 5
4 4
3
PS D:\Weekly modules\Number Patterns>

```